

Real-Time Text Simplification for Dyslexic Readers

Project Proposal for UCS503P

Submitted to: Paramveer Kaur

Thapar Institute of Engineering and Technology

Suneet Singh Arora, CSED*

Dhruv Kumar Garg, CSED[†]

August 14, 2026

1 Higher-order goal

... secondary framing

Help dyslexic and struggling readers comprehend dense text more easily by developing a real-time text simplification and accessibility tool. The system rewrites pasted text or uploaded document content into shorter, simpler sentences using natural language processing while preserving the original meaning, and renders it with dyslexia-friendly typography. It also provides a read along text-to-speech mode with word-level highlighting and lets users compare the simplified version against the original, keeping the reader in control of how aggressively the text is simplified.

2 Time-to-value

... primary heuristic

- We will deliver early value by developing the core real-time text simplification pipeline. The first version will accept complex text, identify difficult words and sentence structures, and generate simpler alternatives for dyslexic readers.
- The initial iteration will focus on text preprocessing, readability analysis, difficult word detection, and NLP-based simplification. Once stable, additional features such as personalization and adaptive difficulty adjustment will be introduced.
- Future iterations will add context-aware simplification, user-specific reading profiles, text-to-speech support, and accessibility-focused analytics.
- Success will be measured by demonstrating a working system that simplifies text in real time, improves readability, and helps dyslexic readers better understand complex content.

*Roll No: 1024030174, Email: sarora2_be24thapar.edu

[†]Roll No: 1024030175, Email: dgarg1_be24thapar.edu

3 Problem Statement

Dyslexic readers often face challenges while processing complex written content due to difficulties with word recognition, sentence comprehension, and reading fluency. Existing accessibility tools may provide basic text adjustments but often lack intelligent simplification that preserves meaning while improving readability. The main problems are:

- **Complex text comprehension:** Long sentences, difficult vocabulary, and complex grammatical structures can increase cognitive load and make reading challenging for dyslexic individuals.
- **Limited text adaptation:** Existing solutions often provide formatting changes but do not dynamically simplify text based on linguistic complexity and reader needs.
- **Loss of meaning during simplification:** Basic text simplification approaches may reduce complexity but fail to preserve the original context and intent of the content.
- **Lack of real-time assistance:** Many accessibility tools require manual processing of text, limiting their usefulness for continuous reading experiences across digital platforms.

Why this matters now (contextual relevance): With increasing amounts of digital content being consumed daily, dyslexic readers need intelligent accessibility solutions that can reduce reading difficulty while preserving the original meaning of text. A real-time AI-based text simplification system can improve reading comprehension, provide personalized assistance, and create a more inclusive digital reading experience without replacing the reader's ability to engage with the original content.

4 Proposed Solution

... Software Proposition

4.1 Overview

Build a **Real-Time Text Simplification System for Dyslexic Readers** with the following components:

- **Text input and preprocessing module:** accepts digital text input and performs preprocessing operations such as sentence segmentation and linguistic analysis to prepare content for simplification.
- **Text complexity analysis module:** identifies difficult words, complex sentence structures, and readability factors that may create challenges for dyslexic readers.
- **AI-based simplification engine:** uses natural language processing techniques to generate simpler alternatives while preserving the original meaning and context of the text.
- **Real-time assistance module:** provides quick text simplification after user input, reducing the effort required to manually rewrite complex content.
- **Personalization module:** adapts simplification levels based on user preferences and reading requirements to provide a more effective reading experience.
- **Accessibility support module:** integrates features such as improved readability formatting and optional text-to-speech support to enhance content accessibility.

4.2 Core workflow

1. User provides the text content through the system interface. The input text is processed and prepared through preprocessing steps such as sentence segmentation and tokenization.
2. The system analyzes the text to identify linguistic complexity, difficult vocabulary, and sentence structures that may be challenging for dyslexic readers.
3. The simplification engine applies NLP-based techniques to generate easier-to-understand alternatives while maintaining the original meaning and context.
4. The simplified text is presented to the user in real time, along with improved readability features to support easier comprehension.
5. The system can adapt simplification based on user preferences, allowing different levels of text complexity reduction for individual reading needs.
6. Additional accessibility features such as text-to-speech and readability enhancements can be integrated to provide a more inclusive reading experience.

The overall workflow is:

Input Text → *Text Processing* → *Complexity Analysis* → *AI Simplification* → *Simplified Output* → *Improved Reading Experience*.

4.3 Operational constraints for an educational, resource-limited setting

- Ensure the core text simplification pipeline can run on standard computing resources without requiring specialized hardware.
- Prioritize simplification accuracy, meaning preservation, readability improvement, and real-time response over unnecessarily complex model architectures.
- Use lightweight NLP models and optimization techniques to maintain efficient processing and low latency during text simplification.
- Minimize storage of user-provided text and personal reading data to reduce privacy and security risks.
- Keep the system lightweight and suitable for local development or deployment on modest cloud infrastructure.

5 Solution Approach

... *Engineering focus*

This is an engineering-focused project, so the emphasis is on building a practical accessibility solution for dyslexic readers using established natural language processing, machine learning, and software engineering techniques rather than developing a novel language model. The system focuses on real-time text analysis and simplification to improve readability while preserving the original meaning of the content.

5.1 Text Simplification

... *practical NLP approach*

Text simplification will use a combination of linguistic analysis and NLP-based techniques:

- Preprocess input text to identify sentence boundaries, word complexity, and linguistic patterns that affect readability.

- Analyze text difficulty using readability metrics, vocabulary complexity, and sentence structure analysis.
- Generate simplified text using NLP-based transformation techniques while preserving the original meaning and context.
- Establish a baseline simplification approach and evaluate improvements using readability metrics and human evaluation where possible.
- Combine complexity analysis and simplification methods to produce clearer, more accessible text for dyslexic readers.
- Provide simplified output as reading assistance while keeping the user in control of the original content and interpretation.

5.2 AI-assisted text adaptation

...generative assistance

Generative AI will be used selectively for improving text accessibility:

- Generate simplified versions of complex sentences while preserving the original meaning and context.
- Provide alternative explanations for difficult words, phrases, and concepts to improve understanding.
- Adapt text complexity based on user preferences and reading requirements.
- Ensure generated simplifications assist reading comprehension and do not alter the intended meaning of the original content.

5.3 Web architecture

...web-based solution

A three-tier web architecture with an integrated NLP pipeline for real-time text simplification:

- **Presentation layer:** React-based user interface for entering text, viewing simplified output, adjusting simplification preferences, and accessing accessibility features.
- **Application layer:** FastAPI backend manages text processing requests, user preferences, simplification workflows, and communication between the interface and NLP services.
- **NLP and data layer:** Python-based NLP services perform text analysis, complexity detection, and simplification generation, while optional storage manages user preferences, processing history, and system configuration data.

5.4 Data Management

...lightweight user data storage

The system will use lightweight storage for maintaining user preferences and optional interaction history while minimizing unnecessary retention of user-generated content.

The database may contain entities such as:

- **Users:** stores user identifiers and accessibility preferences such as preferred simplification level, reading settings, and interface preferences.
- **Text processing history:** stores optional records of processing activity, timestamps, and references to previous simplification sessions without retaining unnecessary user-generated content.

- **Language analysis data:** stores extracted readability metrics, text complexity scores, and identified difficult words for evaluation and system improvement.
- **System configuration:** stores application settings, model parameters, and simplification preferences required for consistent system operation.

The system will avoid unnecessary storage of user-provided text and retain only essential metadata required for user preferences, system operation, and optional history management.

5.5 Development and Testing

Git/GitHub will be used for version control, while automated tests will verify important application, NLP, and text-processing functionality.

- **Version control:** Git and GitHub will be used to track development changes, manage stable versions, and maintain a structured codebase.
- **Backend testing:** automated tests will verify API endpoints, input validation, user preference handling, and core application workflows.
- **NLP testing:** tests will evaluate text preprocessing, complexity analysis, simplification generation, and preservation of original meaning.
- **Performance testing:** the system will be evaluated for response time and processing efficiency to ensure real-time text simplification.
- **Reproducibility:** dependency files and consistent project structure will be maintained so that the application and NLP pipeline can be reliably recreated.

6 Evaluation Criteria

... measurable and attributable

We define success using measurable criteria covering simplification quality, readability improvement, meaning preservation, and system performance.

6.1 Primary evaluation metric

Text simplification quality: measure how effectively the system reduces text complexity while maintaining the original meaning and improving readability for dyslexic readers.

- **Metrics:** evaluate readability improvement using metrics such as Flesch Reading Ease, sentence complexity reduction, vocabulary difficulty reduction, and human evaluation of simplified outputs.
- **Attribution:** evaluate performance on a test set of complex texts by comparing original and simplified versions based on readability improvement and meaning preservation.

6.2 Secondary evaluation metrics

- **Meaning preservation:** evaluate whether simplified text retains the original intent and information without introducing incorrect changes or losing important details.
- **Vocabulary simplification quality:** measure whether complex words and phrases are replaced with simpler alternatives that remain appropriate for the context.
- **Sentence structure improvement:** evaluate the reduction of complex sentence patterns, sentence length, and grammatical difficulty after simplification.

- **Readability improvement:** compare readability scores between original and simplified text to determine whether the system effectively reduces reading difficulty.
- **User usability:** assess whether dyslexic readers can easily interact with the system, understand the simplified output, and adjust accessibility preferences.
- **Processing efficiency:** measure the time required to analyze and simplify text to ensure the system provides real-time assistance.

6.3 Pilot validation plan

... *high-level*

- Evaluate the system on a fixed collection of complex texts covering different domains, reading levels, and types of linguistic difficulty.
- Compare original and simplified outputs using readability metrics to measure reduction in text complexity and improvement in accessibility.
- Manually review simplified texts to evaluate meaning preservation, vocabulary choices, and whether the generated content remains faithful to the original text.
- Conduct a small usability evaluation with student users to determine whether the simplified output improves readability and whether the interface is easy to use.
- Record common simplification errors, meaning changes, and processing delays to refine the NLP pipeline before the final evaluation.

7 Scalability

... *with foresight*

The proposed system should remain practical as the volume of text, users, and simplification requests increases.

1. **Scalable (theoretically):** The system should support increasing simplification workload by:
 - keeping text extraction, NLP, simplification, and presentation components modular,
 - allowing simplified segments to be cached and reused for repeated or overlapping text,
 - separating text ingestion from simplification logic so that new simplification models or rules can be swapped in without reprocessing the entire pipeline.
2. **Operationally deployable (practically):** The system should remain suitable for deployment on common cloud infrastructure by:
 - separating the frontend, backend, and NLP/simplification processing components,
 - using lightweight persistent storage for optional user preferences, simplification history, and analytics,
 - containerizing the application components for reproducible deployment,
 - allowing more computationally intensive simplification workloads (ML/LLM-based rewriting) to be scaled independently if usage increases.

8 Engine Availability Heuristic

A mature set of tools and libraries is available to implement the proposed system as a web-based text simplification platform:

- **Frontend framework:** React for the text input interface, simplification output view, and readability comparison display.
- **Backend framework:** FastAPI for application endpoints, validation, asynchronous requests, and integration between the frontend, database, and NLP components.
- **NLP and machine learning:** Python, spaCy/NLTK, and readability analysis libraries for sentence segmentation, tokenization, lexical analysis, and readability scoring. Transformer-based models or LLM APIs may be used for advanced semantic-aware simplification.
- **Generative AI:** Gemini API or equivalent models may be used for optional context-aware sentence rewriting and advanced simplification while maintaining a rule-based fallback approach.
- **Database:** Lightweight relational storage for optional user preferences, simplification history, and readability analytics.
- **Model and data processing:** Python-based services for text extraction, sentence segmentation, lexical substitution, and readability scoring.
- **Testing and version control:** pytest, Git, and GitHub for application testing and source-code management.
- **Deployment:** Docker for reproducible local and cloud deployment of the application components.

We will use these established components to focus on simplification quality, readability improvement, real-time performance, and accessibility rather than developing infrastructure or language models from scratch.

9 Project Scope and Deliverables

... iteration-friendly

9.1 Initial Deliverable

... first iteration

- Text input support via paste and file upload.
- Preprocessing pipeline: sentence segmentation and readability scoring (Flesch-Kincaid, SMOG).
- Rule-based lexical simplification using a complex-to-simple word substitution dictionary.
- Baseline syntactic simplification for long or complex sentences.
- Initial dyslexia-friendly presentation layer (adjustable font, spacing, line height, background tint).
- Side-by-side display of original text, simplified text, and readability score comparison.
- Evaluation of simplification quality on a fixed validation set of sample texts.

9.2 Subsequent Deliverables

- Improved simplification pipeline using ML/LLM-based rewriting compared against the rule-based baseline.
- Real-time performance optimization through debounced/incremental processing and caching of simplified segments.
- Meaning-preservation and fluency evaluation of AI-generated simplifications against source text.
- Adjustable simplification intensity and per-user calibration of reading level.
- Text-to-speech integration for simplified output.
- Future extension: browser-based overlay for simplifying live web-page content beyond pasted text.
- Readability analytics dashboard showing before/after scores and simplification statistics across sessions.
- Final web application (frontend + backend) with deployment, testing, documentation, GitHub repository, and final demonstration.

10 Risks and Mitigations

- **Simplification distorting meaning:** aggressive lexical or syntactic rewriting may drop nuance or alter the original intent of the text; evaluate simplified output against the source for meaning preservation, and let users toggle simplification intensity.
- **Poor readability improvement:** rule-based substitutions may not meaningfully reduce reading difficulty for all users; compare multiple approaches (rule-based vs. ML-based), evaluate against readability metrics (Flesch-Kincaid, SMOG) on held-out text samples, and retain manual review during testing.
- **Hallucinated or unfaithful rewrites:** generative-AI-based simplification may introduce content not present in the original text; ground prompts in the extracted source text and require outputs to remain traceable to it.
- **Text extraction issues:** varied input sources (PDFs, web pages, pasted text) may result in incomplete or malformed text extraction; validate extracted content and provide fallback handling for unsupported formats.
- **Real-time performance constraints:** transformer-based or LLM-based simplification may introduce latency that undermines the “real-time” experience; use debounced/incremental processing, caching of previously simplified segments, and lightweight models where practical.
- **Accessibility validity:** presentation choices (fonts, spacing, color) must align with established dyslexia-friendly guidelines rather than assumptions; reference standards such as the British Dyslexia Association style guide and validate with user feedback where possible.
- **External API dependency:** generative-AI-based simplification features may depend on external API availability or usage limits; ensure a rule-based fallback simplifier can operate independently of external services.

- **Data privacy:** user-submitted text may contain sensitive personal or academic content; minimize retained data, avoid unnecessary storage, and use publicly available or synthetic text for development and testing.
- **Scope creep:** integrating browser-based content support, TTS, per-user calibration, and analytics dashboards may exceed the project’s academic timeframe; prioritize the core text-in → simplified-text-out pipeline before optional features.

11 Summary

This project proposes a real-time text simplification platform designed to help dyslexic readers process written content more easily through pasted text and uploaded documents, with future support for browser-based content. The system combines NLP-based readability analysis, lexical and syntactic simplification, and optional generative AI to rewrite complex sentences into plain language, flag hard-to-read passages, adapt formatting to dyslexia-friendly standards, and provide readability analytics. Development will follow an incremental approach, beginning with text ingestion and readability scoring before progressing to rule-based and ML-driven simplification, real-time performance optimization, adaptive presentation controls, and personalized simplification calibration. Success will be measured through simplification accuracy, readability-score improvement, processing latency, output fluency and meaning preservation, and system usability. The platform is intended as an assistive reading tool that reduces cognitive load for dyslexic users while keeping the original meaning of the text intact and under the user’s control.